Photo Upload API Documentation

Overview

The Photo Upload API allows frontend applications to upload images and documents for different entity types in the system. The API supports uploading multiple files in a single request and automatically saves the URLs to the corresponding database fields.

Base Endpoint

```
POST /api/v1/uploads/photos
```

Authentication

All requests require authentication. Include the JWT token in the Authorization header:

```
Authorization: Bearer <your_jwt_token>
```

Request Format

The API accepts `multipart/form-data` requests with the following fields:

Field	Type	Required	Description
`entity_id`	integer	Yes	The ID of the target entity (apartment, part, sale, or rental contract)
`entity_type`	string	Yes	Type of entity: `part`, `rent`, `sale`, or `rental_contract`
`document_type`	string	Conditional	Required only for `rental_contract`. Must be `contract` or `customer_id`. Optional for other types.
`files`	file[]	Yes	One or more image/document files to upload

Supported Entity Types

1. Apartment Rent (`entity_type: "rent"`)

Purpose: Upload photos for rental apartments.

Required Fields:

- `entity_id`: The apartment rent ID
- `entity_type`: `"rent"`
- `files`: One or more image files

Steps:

1. User selects one or more image files from their device
2. Get the apartment rent ID from your current context (form, route params, or state)
3. Create a FormData object
4. Append `entity_id` with the apartment ID value
5. Append `entity_type` with the string `"rent"`
6. Append each selected file to `files` array in FormData
7. Make POST request to `/api/v1/uploads/photos` with FormData in request body
8. Include Authorization header with Bearer token
9. Set Content-Type header to `multipart/form-data` (browser will set this automatically with boundary)
10. Handle response - on success, the photos are automatically saved to the apartment's `photos_url` field
11. Refresh apartment data or update local state to display new photos

Response:

- Returns list of uploaded files with their URLs
- All URLs are automatically saved to apartment's `photos_url` JSON array field
- Existing photos are preserved, new ones are appended

---

2. Apartment Sale (`entity_type: "sale"`)

Purpose: Upload photos for apartments for sale.

Required Fields:

- `entity_id`: The apartment sale ID
- `entity_type`: `"sale"`
- `files`: One or more image files

Steps:

1. User selects one or more image files from their device
2. Get the apartment sale ID from your current context
3. Create a FormData object
4. Append `entity_id` with the apartment sale ID value
5. Append `entity_type` with the string `"sale"`
6. Append each selected file to `files` array in FormData
7. Make POST request to `/api/v1/uploads/photos` with FormData in request body
8. Include Authorization header with Bearer token
9. Handle response - photos are automatically saved to apartment's `photos_url` field
10. Refresh apartment data to show new photos

Response:

- Returns list of uploaded files with their URLs
- All URLs are automatically saved to apartment's `photos_url` JSON array field

---

3. Apartment Part (`entity_type: "part"`)

Purpose: Upload photos for individual apartment parts/studios.

Required Fields:

- `entity_id`: The apartment part ID
- `entity_type`: `"part"`
- `files`: One or more image files

Steps:

1. User selects one or more image files from their device
2. Get the apartment part ID from your current context
3. Create a FormData object
4. Append `entity_id` with the apartment part ID value
5. Append `entity_type` with the string `"part"`
6. Append each selected file to `files` array in FormData
7. Make POST request to `/api/v1/uploads/photos` with FormData in request body
8. Include Authorization header with Bearer token
9. Handle response - photos are automatically saved to part's `photos_url` field
10. Refresh part data to show new photos

Response:

- Returns list of uploaded files with their URLs
- All URLs are automatically saved to part's `photos_url` JSON array field

---

4. Rental Contract (`entity_type: "rental_contract"`)

Purpose: Upload contract documents or customer ID documents for rental contracts.

Required Fields:

- `entity_id`: The rental contract ID
- `entity_type`: `"rental_contract"`
- `document_type`: `"contract"` or `"customer_id"` (required)
- `files`: One or more document/image files

Document Types:

- `"contract"`: Uploads contract documents, saves URL to `contract_url` field
- `"customer_id"`: Uploads customer ID documents, saves URL to `customer_id_url` field

Steps for Contract Document:

1. User selects one or more document/image files (contract PDF, scanned contract, etc.)
2. Get the rental contract ID from your current context
3. Create a FormData object
4. Append `entity_id` with the rental contract ID value

5. Append `entity_type` with the string `"rental_contract"`
6. Append `document_type` with the string `"contract"`
7. Append each selected file to `files` array in FormData
8. Make POST request to `/api/v1/uploads/photos` with FormData in request body
9. Include Authorization header with Bearer token
10. Handle response - first file's URL is automatically saved to contract's `contract_url` field
11. Refresh contract data to show new document URL

Steps for Customer ID Document:

1. User selects one or more document/image files (ID front, ID back, etc.)
2. Get the rental contract ID from your current context
3. Create a FormData object
4. Append `entity_id` with the rental contract ID value
5. Append `entity_type` with the string `"rental_contract"`
6. Append `document_type` with the string `"customer_id"`
7. Append each selected file to `files` array in FormData
8. Make POST request to `/api/v1/uploads/photos` with FormData in request body
9. Include Authorization header with Bearer token
10. Handle response - first file's URL is automatically saved to contract's `customer_id_url` field
11. Refresh contract data to show new document URL

Important Notes:

- Multiple files can be uploaded, but only the first file's URL is saved to the database field
- All files are saved to storage, but the database field stores a single URL
- If you upload again with the same `document_type`, it will replace the previous URL

Response:

- Returns list of all uploaded files with their URLs
- Includes `url_field_updated` indicating which field was updated (`contract` or `customer_id`)
- Includes `url_saved` showing the URL that was saved to the database

---

Response Format

Success Response (200 OK)

For Apartments and Parts:

```
{
  "entity_id": 123,
  "entity_type": "rent",
  "count": 2,
  "files": [
    {
      "key": "rent/123/abc123.jpg",
      "url": "/uploads/rent/123/abc123.jpg"
    },
```

```
    {
      "key": "rent/123/def456.png",
      "url": "/uploads/rent/123/def456.png"
    }
  ],
  "folder_key": "rent/123/",
  "saved_to_db": true
}
```

For Rental Contracts:

```
{
  "entity_id": 456,
  "entity_type": "rental_contract",
  "document_type": "contract",
  "count": 2,
  "files": [
    {
      "key": "rental_contract/456/xyz789.pdf",
      "url": "/uploads/rental_contract/456/xyz789.pdf"
    },
    {
      "key": "rental_contract/456/abc123.pdf",
      "url": "/uploads/rental_contract/456/abc123.pdf"
    }
  ],
  "folder_key": "rental_contract/456/",
  "saved_to_db": true,
  "url_field_updated": "contract",
  "url_saved": "/uploads/rental_contract/456/xyz789.pdf"
}
```

Error Responses

400 Bad Request - Invalid entity_type:

```
{
  "detail": "entity_type must be one of: part, rent, rental_contract,
sale"
}
```

400 Bad Request - Missing document_type for rental_contract:

```
{
  "detail": "document_type is required for rental_contract. Must be
'contract' or 'customer_id'"
}
```

400 Bad Request - Invalid document_type:

```
{
  "detail": "document_type must be 'contract' or 'customer_id' for
rental_contract"
}
```

400 Bad Request - No files provided:

```
{
  "detail": "No valid files provided"
}
```

404 Not Found - Entity not found:

```
{
  "detail": "Target entity not found"
}
```

or

```
{
  "detail": "Rental contract not found"
}
```

401 Unauthorized - Missing or invalid token:

```
{
  "detail": "Could not validate credentials"
}
```

403 Forbidden - Insufficient permissions:

```
{
  "detail": "Admin access required."
}
```

500 Internal Server Error:

```
{
  "detail": "Failed to save uploaded files: <error message>"
}
```

File Storage Details

Local Storage (Default)

- Files are stored in: `uploads/{entity_type}/{entity_id}/`
- URLs are relative: `/uploads/{entity_type}/{entity_id}/{filename}`
- Files are accessible via static file serving at `/uploads/...`

Remote Storage (S3 - When Configured)

- Files are stored in S3 bucket with same structure
- URLs are absolute: `https://your-bucket.s3.amazonaws.com/{entity_type}/{entity_id}/{filename}`
- Configuration is handled server-side via environment variables

---

Frontend Implementation Checklist

Before Making Request:

- [ ] User has selected one or more files
- [ ] You have the correct entity ID
- [ ] You know which entity type to use
- [ ] For rental contracts, you know which document type
- [ ] User is authenticated and you have a valid JWT token

During Request:

- [ ] Create FormData object
- [ ] Append all required fields
- [ ] Append all selected files
- [ ] Set Authorization header
- [ ] Handle loading state (show spinner/progress)

After Request:

- [ ] Check response status
- [ ] On success: Extract file URLs from response
- [ ] On success: Update local state/UI with new photos
- [ ] On success: Optionally refresh entity data from API
- [ ] On error: Display error message to user
- [ ] On error: Handle specific error cases (404, 403, etc.)

Best Practices

1. **File Validation:** Validate file types and sizes on frontend before upload
2. **Progress Indicators:** Show upload progress for better UX
3. **Error Handling:** Display user-friendly error messages
4. **State Management:** Update local state after successful upload
5. **Multiple Files:** Allow users to select multiple files at once
6. **File Preview:** Show preview of selected files before upload
7. **Retry Logic:** Allow users to retry failed uploads
8. **Loading States:** Disable submit button during upload

Notes

- All files are saved to storage, even if multiple files are uploaded for rental contracts
- For rental contracts, only the first file's URL is saved to the database field
- For apartments and parts, all file URLs are appended to the `photos_url` array
- Existing photos are preserved when uploading new ones (for apartments and parts)
- Uploading to rental contract fields will replace the previous URL
- File names are automatically sanitized and unique IDs are generated
- The API handles duplicate URLs and deduplicates them automatically